

数组

Julia基础语法

苏州同元软控信息技术有限公司

2024年2月27日



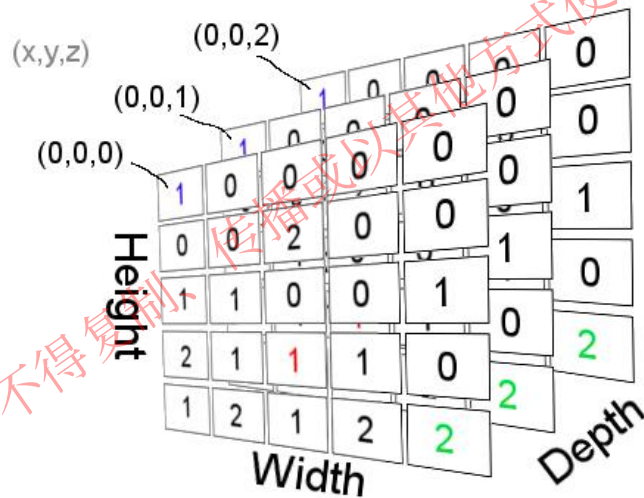
TONGYUAN
Software and Control

数组

数组(Array): 用于容纳同种类型的元素。

Julia支持多维数组:

- 1维数组：向量，如： $[1, 2, 3]$
- 2维数组：矩阵，如： $\begin{bmatrix} 1 & 2 & 3 \\ 2 & 1 & 2 \\ 3 & 2 & 1 \end{bmatrix}$
-
- n维数组



容器类型	元素类型	元素顺序	元素可变性	形式
元组 (Tuple)	任意类型	有序	不可变	(a,b,c)
字典 (Dict)	任意类型的键值对	无序	可变	Dict(a => n, b => m)
集合 (Set)	同种类型 (互不相等)	无序	可变	Set((a,b,c))
数组 (Array)	相同类型 (允许不同类型)	有序	可变	[a, b, c]

四种容器对比

目录

1、数组构造与拼接

2、数组索引与切片

3、数组运算

1. 数组构造与拼接

向量构造与拼接

常用构造形式:

1. `[a,b,c,d]`

```
julia> a = [1,2,3]
3-element Vector{Int64}:
 1
 2
 3
```

2. `[a;b;c;d]`

```
julia> b = [1;2;3]
3-element Vector{Int64}:
 1
 2
 3
```

```
julia> c = Float64[1;2] #构造数组同时约束数值类型
2-element Vector{Float64}:
 1.0
 2.0
```

说明:

- 两种表示形式等价，但是分号表示为拼接
- Julia 的一维向量为列向量

向量拼接: 使用分号、`cat`和`vcat`

```
julia> c = [1:2;4:5]
2-element Vector{UnitRange{Int64}}:
 1:2 4:5
```

```
julia> c = [1:2;4:5]
10-element Vector{Int64}:
 1
 2
 4
 5
```

```
julia> [a,b]
2-element Vector{Vector{Int64}}:
 [1, 2, 3, 4, 5]
 [1, 2, 3, 4, 5]
```

说明:

- 使用逗号不会再解析单独数组元素
- 逗号不能用于拼接
- 向量中的元素可以是任意类型

1. 数组构造与拼接

向量构造与拼接

列表构造

1. `start:step:stop`

- `step`可缺省，缺省时`step=1`

2. `range()`

- 形参为`start`、`step`、`stop`和`length`中任选三个元素
- 默认形参为`start`, `length`, `stop`

```
julia> 1:2  
1:2
```

```
julia> e=range(1,3,3)  
1.0:1.0:3.0
```

```
julia> f=range(1,step=1,length=2)  
1:1:2
```

列表构造向量

1. `[a:b...]`或`[a:b]`

```
julia> a = [1:3...]  
3-element Vector{Int64}:  
 1  
 2  
 3
```

2. `collect(r)`

- `r`为列表

```
julia> b = collect(1:2:3)  
2-element Vector{Int64}:  
 1  
 3
```

说明:

- 列表的运算方式与向量一致
- 列表可以避免不必要的内存分配，效率更高

1. 数组构造与拼接

矩阵构造与拼接

常用构造形式:

1. `[a b;c d]`

```
julia> a = [1 2 3]
1×3 Matrix{Int64}:
 1  2  3
```

```
julia> a1=[1;; 2;; 3;; 4]
1×4 Matrix{Int64}:
 1  2  3  4
```

```
julia> b = [1 2;3 4]
2×2 Matrix{Int64}:
 1  2
 3  4
```

2. `[a b
c d]`

```
julia> c = [1 2
            3 4]
2×2 Matrix{Int64}:
 1  2
 3  4
```

说明:

- 分号、换行符表示纵向(行, 第一维度)拼接
- 空格、双分号表示横向(列, 第二维度)拼接
- 空格的优先级高于分号

向量拼接为矩阵: 使用**空格**、**双分号**、**cat**和**hcat**

```
julia> a = [1,2]; b = Int[3,0,4]; #b中元素数值类型为Int
```

```
julia> [a b]
2×2 Matrix{Int64}:
 1  3
 2  4
```

```
julia> [a;;b]
2×2 Matrix{Int64}:
 1  3
 2  4
```

综合拼接: 空格的优先级高于分号;

使用双分号时, 先纵向拼接再横向拼接

```
julia> [zeros(Int, 2, 2) ; [3 4] ;; [1, 2] ; 5]
3×3 Matrix{Int64}:
 0  0  1
 0  0  2
 3  4  5
```

1. 数组构造与拼接

多维构造与拼接

常用构造形式： 三维数组的第三维度使用三分号连接形成
四维数组的第四维度使用四分号连接形成

.....

```
julia> [1; 2;; 3; 4;; 5; 6;;;
        7; 8;; 9; 10;; 11; 12]
2×3×2 Array{Int64, 3}:
[:, :, 1] =
 1 3 5
 2 4 6

[:, :, 2] =
 7 9 11
 8 10 12
```

```
julia> [1 3 5
        2 4 6;;;
        7 9 11
        8 10 12]
2×3×2 Array{Int64, 3}:
[:, :, 1] =
 1 3 5
 2 4 6

[:, :, 2] =
 7 9 11
 8 10 12
```

```
julia> [[1 2;;; 3 4];;; [5 6];;; [7 8]]
1×2×2×2 Array{Int64, 4}:
[:, :, 1, 1] =
 1 2

[:, :, 2, 1] =
 3 4

[:, :, 1, 2] =
 5 6

[:, :, 2, 2] =
 7 8
```

- 说明：**
- 空格的优先级高于任意数量的分号
 - 数量少的分号优先级高于数量多的分号



1. 数组构造与拼接

数组拼接函数

语法	函数	描述
	<code>cat(A, B, dims=k)</code>	沿着第k维拼接数组
<code>[A; B; C; ...]</code>	<code>vcat(A,B,C,...)</code>	纵向拼接
<code>[A B C ...]</code>	<code>hcat(A,B,C,...)</code>	横向拼接
<code>[A B; C D; ...]</code>	<code>hvcat(rows, A, B, C, D)</code>	同时沿垂直和水平方向拼接
<code>[A; C;; B; D;;; ...]</code>	<code>hvncat</code>	同时进行n维拼接，其中分号的数量表示拼接所在的维度

说明:

- 语法与函数拼接等价
- `cat(A, B, dims=1)`等价于`vcat(A, B)`
- `cat(A, B, dims=2)`等价于`hcat(A, B)`
- `rows`为每行元素个数或数组维度的元组

```
julia> a=[1,2];b=[2,3];c=cat(a,b,dims=2)
2×2 Matrix{Int64}:
 1 2
 2 3

julia> a=[1,2];b=[2,3]; d=vcat(a,b)
4-element Vector{Int64}:
 1
 2
 2
 3

julia> hvcat(1,1,2)
2×1 Matrix{Int64}:
 1
 2

julia> hvcat((2,2),1,2,3,4)
2×2 Matrix{Int64}:
 1 2
 3 4

julia> hvncat(2,3,4,5)
1×3 Matrix{Int64}:
 3 4 5
```



1. 数组构造与拼接

特殊数组构造

函数	描述
<code>Array{T}(undef, dims)</code>	一个没有初始化的密集Array
<code>Matrix{T}(l,m,n)</code>	m行n列的单位矩阵
<code>zeros(T, dims)</code>	每个元素均为0的Array
<code>ones(T, dims)</code>	每个元素均为1的Array
<code>true(dims)</code>	每个元素均为true的BitArray
<code>false(dims)</code>	每个元素均为false的BitArray
<code>rand(T, dims)</code>	一个随机Array，元素值是[0,1)半开区间中均匀分布且服从一阶独立分布
<code>randn(T,dims)</code>	一个随机 Array，元素为标准正态分布，服从独立同分布
<code>range(start, stop =stop, length = n)</code>	从start到stop的带有n个线性间隔元素的范围
<code>fill(x, dims)</code>	一个被值x填充的Array

说明:

- T为数据类型
- dims为数组维度

```
julia> a = Array{Float64}(undef,2,1)
2×1 Matrix{Float64}:
 5.4e-323
 0.0

julia> b = Matrix{Bool}(I,2,3)
2×3 Matrix{Bool}:
 1 0 0
 0 1 0

julia> c=true(dims)
1×3 BitMatrix:
 1 1 1

julia> d=rand(1,3)
1×3 Matrix{Float64}:
 0.483502 0.491862 0.613609

julia> e=range(1,3,3)
1.0:1.0:3.0

julia> f=range(1,step=1,length=2)
1:1:2

julia> [f;] #还可以使用[f...]
2-element Vector{Int64}:
 1
 2
```

说明:

- `Array{T}(undef, dims)`可以完全确定数组类型，Array可以改为Vector和Matrix。



1. 数组构造与拼接

特殊数组构造

函数	描述
fill!(A, x)	用值x填充数组A
reshape(A, dims)	一个包含跟A相同数据但维度不同的数组
copy(A)	拷贝A
similar(A, T, dims)	一个与A具有相同类型(密集、稀疏等)的未初始化数组，但具有制定的元素类型和维度，T和dims可缺省
reinterpret(T, A)	与A具有相同二进制数据的数组，但元素类型为T

```
julia> a=[1,1,1]; b=fill!(Vector{Vector{Int}}(undef, 3), a)
3-element Vector{Vector{Int64}}:
 [1, 1, 1]
 [1, 1, 1]
 [1, 1, 1]

julia> c= reshape(a,1,3)
1×3 Matrix{Int64}:
 1 1 1

julia> reinterpret(Bool, 0xff)
true

julia> reinterpret(Bool, 0x00)
false
```

苏州同元软控信息技术有限公司版权所有，未经许可不得复制、传播或以其它方式使用；

侵权必究。



1. 数组构造与拼接

数组类型: `Array{T, N}`, 其中T为元素数据类型, N为数组维度

```
julia> a = [1,2,3]
3-element Vector{Int64}:
 1
 2
 3

julia> a = [1 3 5] #数组可根据赋值改变维度和值
1×3 Matrix{Int64}:
 1 3 5

julia> typeof([1 3 4; 2 4 5; 1 2 4; 4 5 7])
Array{Int64, 3}

julia> arr = [1, "RUNOOB", 2.5, pi] #元素可以为不同类型
4-element Vector{Any}:
 1
 "RUNOOB"
 2.5
 π = 3.1415926535897...
```

注意:

- 数组类型无法体现元素数量
- 数组是可变的, 可改变维度和值
- 数组类型可以根据赋值类型变化而变化
- 别名Vector等价于类型`Array{T,1}`
- 别名Matrix等价于类型`Array{T,2}`
- 数组允许存在不同类型的变量

1. 数组构造与拼接

数组的基本要素：元素数据类型、维度和尺寸

函数	描述
eltype(A)	获取数组的元素数据类型
ndims(A)	获取数组的维度
length(A)	获取数组元素的总数量
size(A,dims)	获取数组的尺寸

说明：

size(A, dims)中的dims表示输出对应维度的尺寸，如果缺省则返回包含各维度尺寸的元组。

```
julia> a=[1 2 3;3 4 5]
2×3 Matrix{Int64}:
 1  2  3
 3  4  5

julia> eltype(a) #获取a中元素的数据类型
Int64

julia> ndims(a) #获取a的维度
2

julia> length(a) #获取a中元素的总数量
6

julia> size(a) #获取a的尺寸
(2, 3)

julia> size(a,2) #获取a第2维的尺寸
3
```

苏州同元软控信息技术有限公司版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。



1. 数组构造与拼接

数组构造其他形式-推导

一般形式: **$A = [F(x,y,...) \text{ for } x=rx, y=ry,...]$**

其中rx和ry为列表或向量

```
julia> a = [n^2 for n in 1:3]
```

```
3-element Vector{Int64}:
```

```
1
```

```
4
```

```
9
```

```
julia> b = [i+j for i in a, j in 1:3]
```

```
3×3 Matrix{Int64}:
```

```
2 3 4
```

```
5 6 7
```

```
10 11 12
```

```
julia> sum(1/n^2 for n in 1:1000)
```

```
1.6439345666815615
```

```
julia> x = (1/n^2 for n in 1:1000) #先生成一个迭代器
```

```
Base.Generator{UnitRange{Int64}, var"#11#12"{var"#11#12"(), 1:1000}}
```

```
julia> sum(x) #在不分配内存的情况下对一个序列进行求和, 提高效率
```

```
1.6439345666815615
```

目录

1. 数组构造与拼接

2. 数组索引与切片

3. 数组运算

2. 数组索引与切片

数组可以通过索引与切片使用或修改数组

一般形式： Julia数组使用方括号A[]进行索引和切片。

`X = A[l_1,l_2,...,l_n]`

说明：

- l_n可以为标量整数、整数数组或其他支持的索引类型
- 数组索引：l_n均为为标量整数时，结果X为数组A中的单个元素
- 数组切片：l_n不全为标量整数时，结果X为数组A中的一个区域数组

```
julia> A = reshape(collect(1:4),(2,2))
2×2 Matrix{Int64}:
 1  3
 2  4
julia> A[1,2];A[3] #数组元素位置从1开始; 支持线性索引
 3
julia> A[1,[1,2]] # [1,2]表示此维度中第1个和第2个量
2-element Vector{Int64}:
 1
 3
julia> A[[1],[1,2]]
1×2 Matrix{Int64}:
 1  3
```

注意：

- 取矩阵中某一行或一列中某一个或某几个值时，取出的值为对应个数的**向量**。

```
julia> A[1,:] #使用取维度中所有量
2-element Vector{Int64}:
 1
 3
julia> A[1,1:2] # 1:2表示此维度中第1个到第2个量
2-element Vector{Int64}:
 1
 3
julia> A[end,2] #end表示此维度中最后1个量
 4
julia> A[[false,true],:] # [false,true]表示此维度中对应位置为true的量
1×2 Matrix{Int64}:
 2  4
julia> A[CartesianIndex(2, 1)] #CartesianIndex(2, 1)表示使用笛卡尔坐标索引
 2
```

2. 数组索引与切片

注意： Julia 不会在赋值语句中自动增长数组。

```
julia> a = rand(3,3)
3×3 Matrix{Float64}:
 0.317134  0.808967  0.912479
 0.241704  0.420218  0.245492
 0.00407314 0.447554 0.506671

julia> a[3,1] #索引第3行第1列的值
0.0040731381120721055

julia> a[4,1] #索引第4行第1列的值不存在
ERROR: BoundsError: attempt to access 3×3 Matrix{Float64} at index [4, 1]
Stacktrace:
 [1] getindex(::Matrix{Float64}, ::Int64, ::Int64)
       @ Base .\array.jl:862
 [2] top-level scope
       @ REPL[3]:1
```

2. 数组索引与切片

数组修改：使用索引与切片的方式改变数组的值

<pre>julia> X = [1, 2] 2-element Vector{Int64}: 1 2 julia> X[1] = 3 #将X的第1个值改为3 3 julia> X 2-element Vector{Int64}: 3 2</pre>	<pre>julia> X = [1, 2] 2-element Vector{Int64}: 1 2 julia> Y = X 2-element Vector{Int64}: 1 2 julia> Y[1]=5 #X的值是否有变化? 5 julia> X #X的值也会跟随Y的值变化 2-element Vector{Int64}: 5 2 julia> pointer(X)==pointer(Y) #地址相同 true</pre>	<pre>julia> X = [1, 2]; Y = copy(X) 2-element Vector{Int64}: 1 2 julia> pointer(X)==pointer(Y) #地址不同 false julia> X == Y #数值相同 true julia> X === Y #地址不同 false julia> Y[1]=5 #X的值是否有变化? 5 julia> X #X的值不会跟随Y的值变化 2-element Vector{Int64}: 1 2</pre>
--	--	--

注意：

- Julia数组在分配给另一个变量时不会被复制。在A = B之后，改变B的元素也会改变A的元素。

2. 数组索引与切片

数组切片：默认情况是复制内存的

```
julia> X = [1, 2, 3, 4]
4-element Vector{Int64}:
 1
 2
 3
 4

julia> Y = X[1:2] #Y取X的第1个和第2个值
2-element Vector{Int64}:
 1
 2

julia> Y[1] = 0 #改变Y的第一个值
0

julia> X[1] #X的第一个值依然为1
1
```

目录

1. 数组构造与拼接

2. 数组索引与切片

3. 数组运算

3. 数组运算

运算符:

类型	表达式	名称	描述
一元运算符	+	一元加法	全等操作
	-	一元减法	将各元素取反
二元运算符	-	减	各元素相减
	+	加	各元素相减
	*	乘	数组乘法 (线性代数)
	/	除	数组除法 (线性代数)
	\	反向除法	数组反向除法 (线性代数)
	^	幂	数组幂运算
比较操作符	==	相等	任意元素不等则为false
	!=	不相等	任意元素不等则为true
	≈	约等	任意元素不约等则为false
	≈/	不约等于	任意元素不约等则为true

```
julia> a=[1,2,3]; b=[2,3,4];

julia> a+b
3-element Vector{Int64}:
 3
 5
 7

julia> a/b
3×3 Matrix{Float64}:
0.0689655 0.103448 0.137931
0.137931 0.206897 0.275862
0.206897 0.310345 0.413793

julia> ans*b
3-element Vector{Float64}:
1.0
2.0
3.0

julia> c=[1+1e-10,2,3];a≈c
true
```



3. 数组运算

数组运算相关函数：

类别	函数	功能描述
矩阵属性	norm	计算范数
	det	矩阵行列式
	rank	矩阵的秩
	tr	对角线元素之和
	cond	逆运算的条件数
	condskeel	相对条件数
	condeig	与特征值有关的条件数
	rref	简化的行阶梯形矩阵 (Gauss-Jordan 消元法)
	subspace	两个子空间之间的角度
线性方程	\	对线性方程组 $Ax = B$ 求解 x
	/	对线性方程组 $xA = B$ 求解 x
	linsolve	对线性方程组求解
	inv	矩阵求逆
	pinv	Moore-Penrose 伪逆
	lscov	存在已知协方差情况下的最小二乘解
	sylvester	求 Sylvester 方程 $AX + XB = C$ 的 X 解

类别	函数	功能描述
特征值与奇异值	eigvals	返回A的特征值
	eigen	计算 A 的特征值分解, 返回一个特征分解对象 F
	svd	计算 A 的奇异值分解 (SVD) 并返回一个 SVD 对象。
	svdvals	按降序返回 A 的奇异值
	ordeig	拟三角矩阵的特征值
	polyeig	多项式特征值问题
	hessenberg	矩阵的 Hessenberg 形式
	schur	Schur 分解
	schur!	Schur 分解
	rsf2csf	将实数Schur形式变换为复数Schur形式
矩阵分解	cdf2rdf	拟三角矩阵的特征值
	lu	LU 矩阵分解
	ldlt	实对称三对角矩阵S的LDLt分解
	cholupdate	Cholesky 分解的秩 1 更新
	qrdelete	从 QR 分解中删除列或行
	qrinsert	将列或行插入 QR 分解
	qrupdate	QR 分解的秩 1 更新
	planerot	Givens 平面旋转

类别	函数	功能描述
矩阵运算	transpose	转置向量或矩阵
	,adjoint	复共轭转置
	sqrt	矩阵平方根
	exp	矩阵指数运算
	log	矩阵对数运算
	kron	Kronecker 张量积
	cross, ×	叉积
	dot, ·	点积
矩阵结构	bandwidth	矩阵的上下带宽
	tril	矩阵的下三角形部分
	triu	矩阵的上三角形部分
	isbanded	确定矩阵是否在特定带宽范围内
	isdiag	确定矩阵是否为对角矩阵
	ishermitian	确定矩阵是 Hermitian 矩阵
	issymmetric	确定矩阵是对称矩阵
	istril	确定矩阵是否为下三角矩阵
	istriu	确定矩阵是否为上三角矩阵

具体函数使用详见《Syslab帮助文档》



3. 数组运算

广播(broadcast): 将数组参数中低维度的参数扩展，使得其与其他维度匹配，且不会使用额外的内存，并将所给的函数逐元素地应用。

一般形式: f(args...)或.运算符

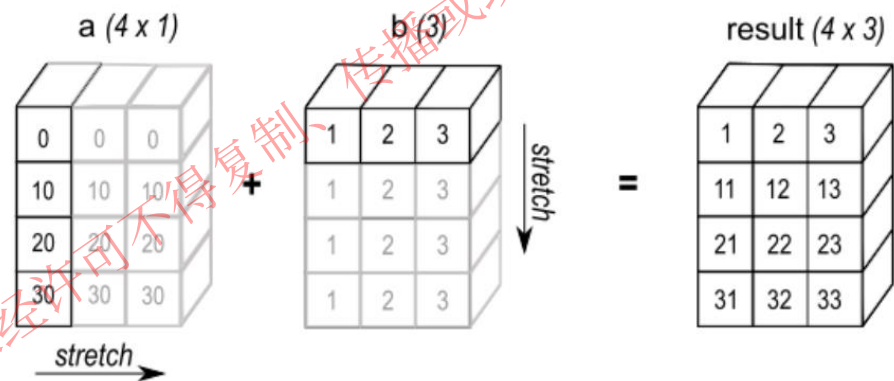
```
julia> a = rand(2,1); A = rand(2,3);

julia> repeat(a,1,3)+A #repeat将a重复形成2×3的矩阵
2×3 Array{Float64,2}:
 1.20813  1.82068  1.25387
 1.56851  1.86401  1.67846

julia> a.+ A #自动扩展a的维度
2×3 Array{Float64,2}:
 1.20813  1.82068  1.25387
 1.56851  1.86401  1.67846

julia> b = rand(1,2)
1×2 Matrix{Float64}:
 0.739175  0.521332

julia> a.+b #自动扩展b的维度
2×2 Matrix{Float64}:
 1.09127  0.873432
 1.04736  0.829514
```



```
julia> c=convert(Float64,1)
1.0

julia> C=convert.(Float64,[1 2]) #使用广播的形式转换矩阵
1×2 Matrix{Float64}:
 1.0  2.0

julia> D=convert.(Float64, (1,2)) #处理元组
(1.0, 2.0)
```

说明:

- 广播不限于数组, 还可以处理标量、元组和其它容器
- 推荐尽量使用广播的形式, 广播能够提高性能



建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

请各位专家指正！

苏州同元软控信息科技有限公司版权所有，未经许可不得转载、传播或以其他方式使用；
版权所有，侵权必究。